

Parallel Computing

Programming with MPI – Part 2

Mariko & Kevin Dunseath

Computational Science and Applications

2025 – 2026

To learn

- how to transfer data between two processes
- possible modes of communication
- trouble shooting

Point-to-Point communications

Definition

Passing a message between two processes:

- one process sends the message
- the other process receives the message

When a call to an MPI send or receive function is made, a request for communication is said to have been "posted"

Different types of point-to-point routines:

- Blocking send / blocking receive
- Non-blocking send / non-blocking receive
- Buffered send (for very large messages)
- Synchronous send

Point-to-Point communications (2)

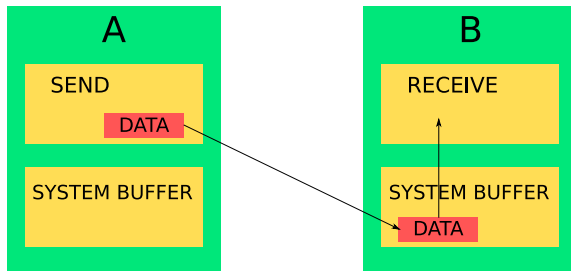
Send operations are not usually perfectly synchronised with their matching receive operations:

- the send may occur before the receive is ready - where is the message stored while the receive is pending?
- multiple sends may arrive at the same process, which can only receive messages one at a time - where are the other messages stored while waiting their turn?

The standard does not specify what to do in these cases, the answer is implementation dependent

Point-to-Point communications: buffering

- Typically, the message is held in a system buffer managed by the MPI Library and not visible to the programmer



- The space of the user's variables and data is called the application buffer

Point-to-Point communications: blocking mode

Blocking send

- Returns control to the program when it is sure it can modify the application buffer without changing the data of the message sent
- Two types
 - synchronous: hand-shaking with receive process to confirm successful reception
 - asynchronous: message is stored in system buffer while waiting to be received. No guarantee that the message has been received

Blocking receive

Returns control to the program only after data has been received

Point-to-Point communications: blocking mode (2)

Blocking send

```
MPI_Send(buffer, len, type, destination, tag, comm)
```

buffer: variable or array containing data to be sent or received.

In C/C++, it must be a pointer or the address of a variable
(`&var`)

len: the number of data items to be transferred

type: the type of data to be transferred

`MPI_INT`, `MPI_DOUBLE`, ...

destination: the rank of the receiving process

tag: a non-negative integer defined by the programmer to label a message. The tags on a send-receive pair must be identical.

comm: communicator, defines the set of processes involved in the communication, usually `MPI_COMM_WORLD`

Point-to-Point communications: blocking mode (3)

Blocking receive

```
MPI_Recv(buffer, len, type, source, tag, comm, status)
```

source: the rank of the sending process

`MPI_ANY_SOURCE` may be used in a receive operation to listen for messages from any other process

tag: a non-negative integer defined by the programmer to label a message. The tags on a send-receive pair must be identical.

`MPI_ANY_TAG` may be used to receive a message regardless of its tag

status: in a receive operation, this contains

- the source of the message `status.MPI_SOURCE`
- its tag `status.MPI_TAG`

It is a structure of type `MPI_Status`

Point-to-Point communications: blocking mode (4)

Standard or blocking mode `Send / Recv`

- will return control to user program when the message buffers used in the argument list of the function call can be safely modified by subsequent statements in the program
- `Send`: the data in the message buffer has been sent or copied into the system buffer
- `Recv`: the data has been copied into the message buffer and can be used

```
double *a, *b;
int ierr;

...

if (rank == source) {
    ierr = MPI_Send(a, len, MPI_DOUBLE, dest, tag,
                    MPI_COMM_WORLD);
    // The array a can be safely updated here
    ...
} else if (rank == dest) {
    ierr = MPI_Recv(b, len, MPI_DOUBLE, source, tag,
                   MPI_COMM_WORLD, &status);
    // The array b can be safely used here
    ...
}
```

MPI program

```
#include <stdio.h>
#include <stdlib.h>
#include "mpi.h"
int main(int argc, char* argv[]) {
    int dest,i,nprocs,rank;
    double x;
    MPI_Status status;
    MPI_Init(NULL,NULL);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);
    MPI_Comm_size(MPI_COMM_WORLD,&nprocs);
    if (rank != 0) {
        dest = 0;
        srand((rank+1)*time(NULL));
        x = (double) rank * rand()/RAND_MAX;
        printf("Process %d has x = %f \n", rank, x);
        MPI_Send(&x,1,MPI_DOUBLE,dest,0,MPI_COMM_WORLD);
    } else {
        for (i=1; i < nprocs; i++) {
            MPI_Recv(&x,1,MPI_DOUBLE,MPI_ANY_SOURCE,0,
                    MPI_COMM_WORLD,&status);
            printf("Process 0 received %f from process %d \n",
                   x, status.MPI_SOURCE);
        }
    }
    MPI_Finalize();
    return 0;
}
```

```
mpic++ communicate.cpp  
mpirun -np 4 ./a.out
```

```
Process 3 has x = 0.680655  
Process 2 has x = 0.727599  
Process 0 received 0.727599 from process 2  
Process 1 has x = 0.495902  
Process 0 received 0.680655 from process 3  
Process 0 received 0.495902 from process 1
```

Point-to-Point communications: non-blocking mode

Non-blocking send and receive

- Return control to the program almost immediately, without waiting for the communication to be completed
- MPI library will perform the communication whenever it can
- It is unsafe to modify the application buffer until it is certain that the data has been transferred

MPI provides routines to check if a communication has been completed successfully

MPI guarantees that messages will not overtake each other so that they will arrive in the order they were sent

Point-to-Point communications: non-blocking mode (2)

Non-blocking send

```
MPI_Isend(buffer, len, type, dest, tag, comm, request)
```

Non-blocking receive

```
MPI_Irecv(buffer, len, type, source, tag, comm,  
          request)
```

request: used by non-blocking routines to label a particular communication. It can be used later on to determine if the communication has completed

Point-to-Point communications: non-blocking mode (3)

Non-blocking mode `Isend` / `Irecv`

- starts or "posts" a send / receive operation
- the user program must explicitly complete the communication later on $\Rightarrow$ requires at least two function calls
- `Isend`: system starts to copy message to system buffer or to its destination
- `IRecv`: system is informed that it can start to copy data in message buffer
- Completion of communication is signaled by the `request` variable in the function call

Point-to-Point communications: synchronous mode

Synchronous send `Ssend`:

- does not complete (*i.e.* return control to user program) until a matching `Recv` has begun to receive the message
- completion indicates that send buffer can be reused **and** that the receiver has reached a certain point in the execution (it has started executing the matching receive)
- does not require system buffering

```
if (rank == 0) {  
    MPI_Ssend(&x,1,MPI_DOUBLE,dest,0,MPI_COMM_WORLD);  
} else if (rank == dest) {  
    MPI_Recv(&x,1,MPI_DOUBLE,0,0 MPI_COMM_WORLD,  
            &status);  
}
```

Point-to-Point communications: buffering (2)

- The system buffer is of finite size and can easily be filled up
- Exceeding the buffer size will cause a program to give erroneous results, or to crash
- To send very large messages, the user must
 - allocate a sufficiently large buffer
 - tell the system to use this buffer to transfer the message
 - call *buffered* versions of the send/receive operations

Point-to-Point communications: buffered mode

Buffered send `Bsend`:

- `Bsend` is local: completion does not depend on the existence of a matching `Recv`
- if there is no matching `Recv` posted, message is copied into a buffer or zone of memory that is explicitly created (allocated) by the user
- `MPI_Buffer_attach` and `MPI_Buffer_detach` are used to associate the buffer with `Bsend`
- the buffer must be sufficiently large to store the message
- only one buffer can be attached at any time

Trouble shooting: deadlocks

```
int a[10], b[10], rank;
MPI_Status status;
...
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
if (rank == 0) {
    MPI_Send(a, 10, MPI_INT, 1, 1, MPI_COMM_WORLD);
    MPI_Send(b, 10, MPI_INT, 1, 2, MPI_COMM_WORLD);
} else if (rank == 1) {
    MPI_Recv(b, 10, MPI_INT, 0, 2, MPI_COMM_WORLD);
    MPI_Recv(a, 10, MPI_INT, 0, 1, MPI_COMM_WORLD);
}
```

- If `MPI_Send` uses buffering, this will work if there is sufficient buffer space
- If `MPI_Send` does not use buffering (blocking until matching receive is posted), neither process can proceed
- Matching the order of the send and receive operations corrects this problem

Trouble shooting: deadlocks (2)

In an exchange, trying to receive data before sending

Trying to receive data when there has been no send

```
if (rank == A) {  
    MPI_Recv(&x, lenx, MPI_INT, B, tag1, MPI_COMM_WORLD, &status);  
    MPI_Send(&y, leny, MPI_INT, B, tag2, MPI_COMM_WORLD);  
} else {  
    MPI_Recv(&y, leny, MPI_INT, A, tag2, MPI_COMM_WORLD, &status);  
    MPI_Send(&x, lenx, MPI_INT, B, tag1, MPI_COMM_WORLD);  
}
```

Since `MPI_Recv` is blocking, this code will certainly hang, since A and B will wait for messages that will not be sent

The program will also hang or crash if there are errors in the parameters, for example a mismatch in the tag, or sending to the wrong destination

Tutorial:

- Write a program where
 - 1 a two dimensional matrix of integers `A[ndim][ndim]` is declared
 - 2 process 0 initializes the matrix and send the values to process 1
 - 3 process 1 prints the matrix
- What happens when all the elements of the matrix are not necessarily consecutive in memory, *i.e.* have been declared as follows?

```
int** a = new int* [ndim];  
for (int i=0; i<ndim; ++i) {  
    a[i] = new int [ndim];  
}
```

- Test the code for other ways of declaring the matrix with all elements consecutive in memory